



MINI PROJECT REPORT
ON
“CARBON CREDIT BANK”

Submitted By

UID No: Khushi Birla (UIT2024928)

UID No: Mugdha Wete (UIT2024938)

UID No: Shivani Nagure (UIT2024939)

UID No: Nehal Katlana (UIT2024941)

DEPARTMENT OF INFORMATION TECHNOLOGY
MKSSS's CUMMINS COLLEGE OF ENGINEERING FOR WOMEN,
PUNE
SAVITRIBAI PHULE PUNE UNIVERSITY
2025 – 2026



DEPARTMENT OF INFORMATION TECHNOLOGY

Certificate

This is to certify that,

UID No: Khushi Birla (UIT2024928)

UID No: Mugdha Wete (UIT2024938)

UID No: Shivani Nagure (UIT2024939)

UID No: Nehal Katlana (UIT2024941)

have completed the Mini Project entitled “**Carbon Credit Bank**”, under my guidance in partial fulfillment of the requirement for the Database Management Laboratory second year in the Department of Information Technology of Savitribai Phule Pune University during the Academic Year 2025 – 2026.

Date :

Place: Pune

Prof. Prajakta Deshpande

Dr. Anagha Kulkarni
(Head, Dept. of Information Technology)

Dr. Madhuri Khambete
Principal

ACKNOWLEDGMENT

We have taken efforts in this project. However, it would not have been possible without the kind support and valuable input from our teachers and guides. I would like to extend my sincere thanks to all of them.

I am highly indebted to Dr. Makarand Velankar, Dr. Milind Kolambe, and Prof. Prajakta Deshpande for their guidance and constant supervision, as well as for providing necessary information regarding the project, and also for their support in completing the project.

Team members names and UID

UID No: Khushi Birla (UIT2024928)

UID No: Mugdha Wete (UIT2024938)

UID No: Shivani Nagure (UIT2024939)

UID No: Nehal Katlana (UIT2024941)

ABSTRACT

Carbon Credit Bank is a transactional Database Management System modeled after a financial banking platform, where the primary asset is carbon credits instead of currency. In the real world, governments impose emission limits on heavy industries — aviation, manufacturing, energy, and transport. Companies that exceed their limits must purchase carbon credits to offset the excess, while cleaner companies can sell their surplus credits. This creates a regulated financial market around environmental accountability.

Our system is the exchange platform that powers this market. A sustainability project — such as a reforestation drive or a solar farm — gets certified by a registry like Verra or Gold Standard. The registry issues a batch of carbon credits tied to that project. Companies can then buy those credits, hold them in a wallet like a bank balance, trade them with other companies, or retire them permanently to offset their own emissions.

Every single action is recorded in an immutable ledger — ensuring nothing can be faked or double-spent. The database enforces market rules directly through CHECK constraints, row-level locking, and referential integrity — making this a serious DBMS application and not merely a CRUD system. The application is built using Python Flask as the web framework, PostgreSQL 16 as the database, and Jinja2 for server-rendered HTML templates.

CONTENTS

Chapter Name	Page Number
1. Introduction	8
2. Scope	9
3. Functional Requirements of the System	10
3.1 Possible users of the system with privileges	
4. Technologies Used	11
4.1 Back end	
4.2 Database	
4.3 Front end	
5. ER Diagram	12
6. Normalization of tables	13
7. Data Dictionary	14
8. Business Logic Implemented	22
9. Testing Documents	23
10. Screenshots	24
Conclusion	31
References	32

LIST OF FIGURES

Sr. No.	Fig.No.	Fig.Title	Page No.
1.	10.1	Dashboard	24
2.	10.2	Login	24
3.	10.3	User Dashboard	25
4.	10.4	User Portfolio	25
5.	10.5	User buys credits	26
6.	10.6	User sells credits	26
7.	10.7	User transaction	27
8.	10.8	Admin dashboard	27
9.	10.9	Admin company list	28
10.	10.10	Admin transaction list	29
11.	10.11	Admin registry list	30
12.	10.12	Admin credits issue	30

LIST OF TABLES

Sr. No.	Fig. No.	Fig. Title	Page No.
1.	6.1	Normalization of tables	13
2.	7.1	Registry	15
	7.2	Company	15
	7.3	Users	16
	7.4	Wallet	16
	7.5	Project	17
	7.6	Credit_batch	18
	7.7	Transaction	19
	7.8	Ownership_history	20
	7.9	Retirement_record	21
3.	9.1	Testing documents	23

1. INTRODUCTION

Carbon credits are certificates that represent one tonne of carbon dioxide either removed from or not released into the atmosphere. Certified registries issue them to sustainability projects and then traded on carbon markets as a financial instrument.

The Carbon Credit Bank is a web-based DBMS application that simulates a transactional carbon credit exchange platform. It enables registered companies to buy certified carbon credits from other organizations, maintain a wallet-based balance, track their portfolio of held credits, list their own credits for sale, and permanently retire credits to offset their emissions. Every transaction is stored in a tamper-proof audit trail, making the system suitable for regulated market environments.

The need for this system arises from the growing complexity of real-world carbon markets, where transparency, auditability, and the prevention of double-spending are critical requirements. This project demonstrates how a well-normalized relational database, combined with proper transactional logic and constraint enforcement, can serve as the foundation for a financially sensitive application.

The primary users of the system are:

1. Companies — organizations that buy, sell, and retire carbon credits
2. Registries — certifying bodies that issue credits to sustainability projects
3. Administrators — users with elevated privileges to view cross-company data

2. SCOPE

The Carbon Credit Bank system covers the following functional scope:

- Company registration and authentication via a session-based login system
- Issuance of carbon credit batches linked to certified sustainability projects
- Buying and selling of credit batches between companies through an atomic transactional workflow
- Portfolio management showing current credit holdings and wallet balances
- Credit retirement workflow with permanent record-keeping
- Full audit trail of every ownership transfer from issuance to retirement

The system is scoped for academic demonstration and proof-of-concept purposes. It does not include real payment gateway integration, regulatory compliance APIs, or external registry verification. Passwords are stored in plain text for demo purposes — production deployment would require hashing and additional security hardening.

The system is designed to run locally on both macOS and Windows machines, making it accessible to all team members for development and demonstration.

3. FUNCTIONAL REQUIREMENTS OF THE SYSTEM

This system will contain all the details of the companies participating in the carbon credit market, and it must be able to allow companies to register themselves and perform transactions involving carbon credits. The system users should be able to view their portfolio, including wallet balance and owned credit batches. The system must provide the functionality to buy and sell carbon credits while ensuring that all transactions are performed atomically and securely.

The system must also maintain a transaction history for each company and allow users to retire credits permanently with proper reason logging. Additionally, the system should maintain an ownership history to track every transfer of carbon credit batches between companies, ensuring transparency and traceability.

The functions performed by the system would be the following:

1. To allow companies to register and log in using a session-based authentication system.
2. To provide a company directory dashboard showing all registered organizations.
3. To display a portfolio view including wallet balance and owned credit batches.
4. To allow companies to buy carbon credits with atomic transactional updates.
5. To allow companies to sell carbon credits by listing owned batches at a defined price.
6. To maintain transaction history specific to the logged-in company.
7. To allow permanent retirement of carbon credits with proper reason logging.
8. To maintain an immutable ownership history for tracking all credit transfers.

3.1 POSSIBLE USERS OF THE SYSTEM WITH PRIVILEGES

The two main groups of users in the Carbon Credit Bank system are administrators and regular users.

3.1.1 ADMIN:

An ADMIN user belongs to a company and has full access to all system functionalities. The admin can log in, view the dashboard, manage the company portfolio, perform buying, selling, and retirement of carbon credits, and view all transaction records. In an extended system, the admin may also access cross-company analytics and reports.

3.1.2 USER:

A USER is a regular system user who can log in, view the company dashboard, and manage their own company portfolio. The user can perform buying, selling, and retirement operations on carbon credits and can view only their own transaction history.

4. TECHNOLOGIES USED

Layer	Technology	Version
Backend	Python / Flask	3.11 / 3.1.3
Database	PostgreSQL	16.13
DB Driver	psycopg2-binary	2.9.11
Templating	Jinja2	3.1.6
Config	python-dotenv	1.2.2
Frontend	HTML5 + CSS3	Server-rendered
Version Control	Git / GitHub	2.50.1

4.1 Backend — Python Flask

Flask is a lightweight WSGI web framework for Python. It provides routing, request handling, session management, and Jinja2 template rendering. We chose Flask for its simplicity and the fact that it requires minimal boilerplate — the entire application logic is visible and understandable, which is appropriate for an academic project. Flask's Blueprint system allows routes to be organized modularly.

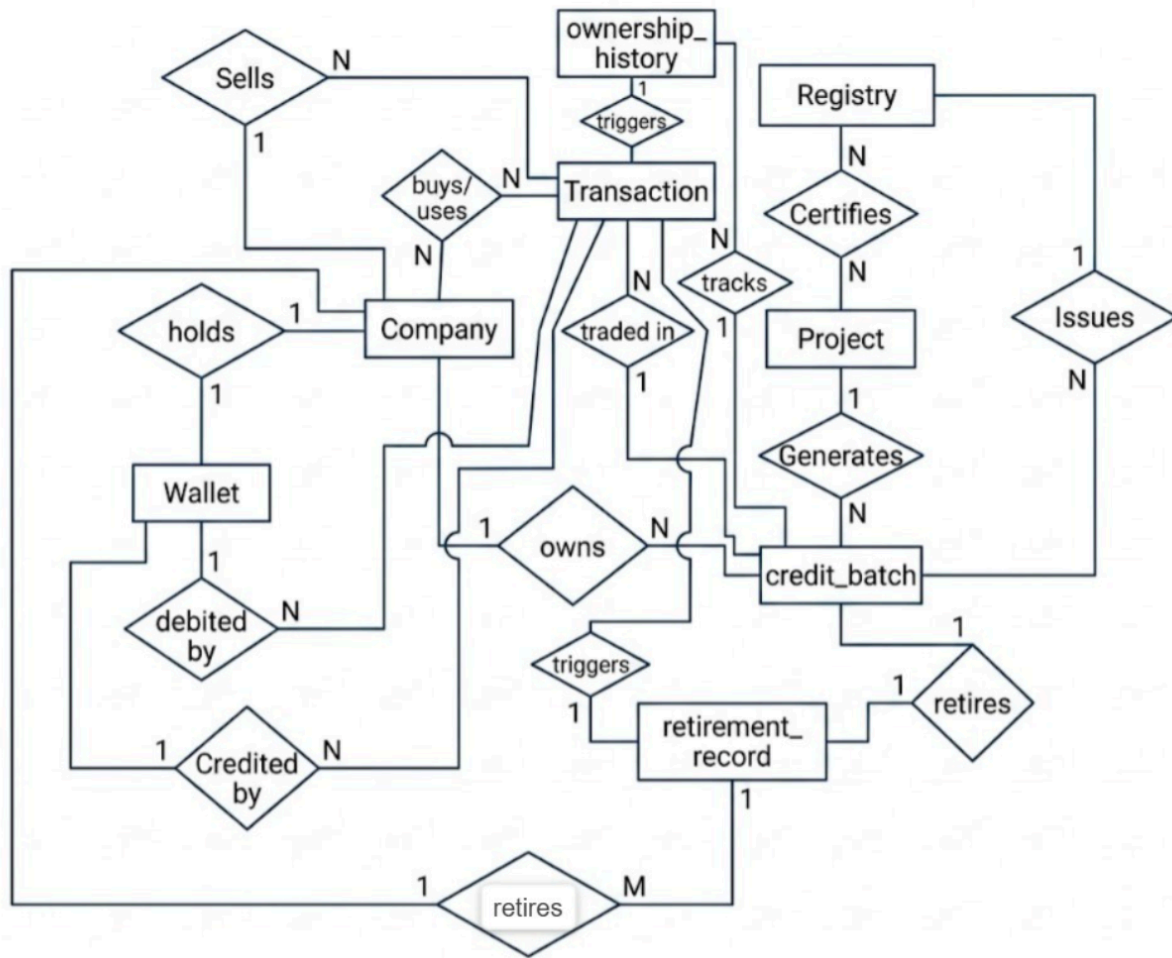
4.2 Database — PostgreSQL

PostgreSQL 16 was chosen over MySQL for three specific reasons relevant to this project. First, CHECK constraints are fully enforced in PostgreSQL — in older versions of MySQL, they are parsed but silently ignored. Second, PostgreSQL natively supports UUID primary keys via the pgcrypto extension, which is superior to auto-increment integers for distributed financial systems. Third, PostgreSQL's row-level locking via SELECT FOR UPDATE provides robust double-spend prevention — critical for a transactional credit system.

4.3 Frontend — HTML5 + CSS3 with Jinja2

The frontend uses server-rendered HTML templates through Flask's Jinja2 integration. All pages extend a common base.html template that provides the navigation bar and page structure. CSS is hand-written with a green color scheme (primary: #1a7a4a) chosen to reflect the environmental domain of the project. No external CSS frameworks are used, keeping the frontend lightweight and dependency-free.

5. ER DIAGRAM



6. NORMALIZATION OF TABLES

Table 6.1: Normalization

Table	Normal Form	Justification
registry	BCNF	All attributes depend solely on registry_id. No partial or transitive dependencies.
company	BCNF	registration_number is a candidate key. No transitive dependencies exist.
users	BCNF	user_id is PK; email is a UNIQUE candidate key. No non-key determines another non-key.
wallet	BCNF	wallet_id is PK; company_id is UNIQUE. 1:1 enforced. No redundancy.
project	BCNF	project_id determines all attributes. registry_id is an FK, not a determinant of other non-keys.
credit_batch	BCNF	batch_id determines all attributes. Multiple FKs represent multi-role participation, not redundancy.
transaction	3NF	transaction_id determines all attributes. buyer_id and seller_id are independent roles, not transitive. total_amount is stored as a snapshot for audit integrity — a justified denormalization.
ownership_history	BCNF	history_id determines all. Immutable insert-only table. No updates allowed.
retirement_record	BCNF	retirement_id determines all. retirement_reason depends only on PK.

7. DATA DICTIONARY

Schema Overview

Table	Primary Key	Foreign Keys	Purpose
registry	registry_id	—	Certifying bodies (Verra, Gold Standard, ACR)
company	company_id	—	Organizations participating in the carbon market
users	user_id	company_id	Login credentials linked to a company
wallet	wallet_id	company_id (UNIQUE)	One USD wallet per company — financial ledger
project	project_id	registry_id	Sustainability projects that generate credits
credit_batch	batch_id	project_id, registry_id, owner_company_id	Tradable carbon credit inventory with quantity and price
transaction	transaction_id	buyer_id, seller_id, batch_id, wallet_debit_id, wallet_credit_id	Every buy/sell/transfer/retirement event
ownership_history	history_id	batch_id, from_company_id, to_company_id, transaction_id	Immutable audit trail of credit batch transfers
retirement_record	retirement_id	batch_id, company_id, transaction_id	Permanent record of retired credits

Table 1: registry

Column	Type	Constraint	Description
registry_id	UUID	PK, DEFAULT gen_random_uuid()	Unique identifier for the registry
name	VARCHAR(200)	NOT NULL, UNIQUE	Registry name (e.g., Verra, Gold Standard)
country	VARCHAR(100)	NOT NULL	Country where the registry is based
accreditation_body	VARCHAR(200)	—	The body that accredits the registry
contact_email	VARCHAR(150)	—	Official contact email
created_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp

Table 2: company

Column	Type	Constraint	Description
company_id	UUID	PK, DEFAULT gen_random_uuid()	Unique identifier for the company
name	VARCHAR(200)	NOT NULL	Company name
registration_number	VARCHAR(100)	NOT NULL, UNIQUE	Official registration number
country	VARCHAR(100)	NOT NULL	Country of incorporation
sector	VARCHAR(100)	—	Industry sector (e.g., ENERGY, AVIATION)
status	VARCHAR(20)	CHECK (ACTIVE/SUSPENDED/DISSOLVED)	Account status
created_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp

Table 3: users

Column	Type	Constraint	Description
user_id	UUID	PK, DEFAULT gen_random_uuid()	Unique identifier for the user
company_id	UUID	FK → company(company_id), NOT NULL	The company this user belongs to
email	VARCHAR(150)	NOT NULL, UNIQUE	Login email address
password	VARCHAR(255)	NOT NULL	Password (plain text for demo; hash in production)
role	VARCHAR(20)	CHECK (ADMIN/USER), DEFAULT 'USER'	User privilege level
created_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp

Table 4: wallet

Column	Type	Constraint	Description
wallet_id	UUID	PK, DEFAULT gen_random_uuid()	Unique identifier for the wallet
company_id	UUID	FK → company(company_id), UNIQUE	One wallet per company (1:1 enforced)
balance	NUMERIC(20,4)	DEFAULT 0, CHECK (balance >= 0)	Current USD balance — cannot go negative
currency	VARCHAR(10)	DEFAULT 'USD'	Currency denomination
updated_at	TIMESTAMP	DEFAULT NOW()	Last balance update timestamp

Table 5: project

Column	Type	Constraint	Description
project_id	UUID	PK, DEFAULT gen_random_uuid()	Unique identifier for the project
registry_id	UUID	FK → registry(registry_id), NOT NULL	Certifying registry
name	VARCHAR(300)	NOT NULL	Project name
type	VARCHAR(100)	CHECK (REFORESTATION/SOLAR/WIND/METHANE_CAPTURE/OCEAN/DIRECT_AIR_CAPTURE/OTHER)	Project category
location	VARCHAR(200)	—	Geographic location
start_date	DATE	—	Project start date
end_date	DATE	—	Project end date
status	VARCHAR(20)	CHECK (ACTIVE/COMPLETE/D/SUSPENDED)	Current project status
created_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp

Table 6: credit_batch

Column	Type	Constraint	Description
batch_id	UUID	PK, DEFAULT gen_random_uuid()	Unique identifier for the credit batch
project_id	UUID	FK → project(project_id), NOT NULL	Source sustainability project
registry_id	UUID	FK → registry(registry_id), NOT NULL	Issuing registry
owner_company_id	UUID	FK → company(company_id), NOT NULL	Current owner company
quantity	INTEGER	NOT NULL, CHECK > 0	Total credits in batch (immutable after issuance)
quantity_available	INTEGER	NOT NULL, CHECK ≥ 0	Remaining tradeable credits
unit_price	NUMERIC(14,4)	—	Listed price per credit in USD
certification_standard	VARCHAR(100)	—	Standard used (VCS, Gold Standard, ACR)
vintage_year	DATE	NOT NULL	Year credits were generated
expiry_date	DATE	—	Date after which credits cannot be traded
status	VARCHAR(20)	CHECK (AVAILABLE/PARTIALLY_SOLD/SOLD_OUT/EXPIRED/RETIRED)	Batch trading status
created_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp

Table 7: transaction

Column	Type	Constraint	Description
transaction_id	UUID	PK, DEFAULT gen_random_uuid()	Unique transaction identifier
buyer_id	UUID	FK → company(company_id)	Buying company (NULL for retirement/issuance)
seller_id	UUID	FK → company(company_id)	Selling company (NULL for retirement)
batch_id	UUID	FK → credit_batch(batch_id), NOT NULL	Credit batch being traded
wallet_debit_id	UUID	FK → wallet(wallet_id)	Buyer's wallet to debit
wallet_credit_id	UUID	FK → wallet(wallet_id)	Seller wallet to credit
quantity	INTEGER	NOT NULL, CHECK > 0	Number of credits in this transaction
price_per_unit	NUMERIC(14,4)	—	Price per credit at time of transaction
total_amount	NUMERIC(20,4)	—	Snapshot total (quantity × price) for audit integrity
transaction_type	VARCHAR(20)	CHECK (BUY/SELL/TRANSFER/ISSUANCE/RETIREMENT)	Type of operation
status	VARCHAR(20)	DEFAULT 'PENDING', CHECK (PENDING/COMPLETED/FAILED/REVERSED)	Transaction outcome
timestamp	TIMESTAMP	DEFAULT NOW()	Exact time of transaction
notes	TEXT	—	Optional remarks or reason

Table 8: ownership_history

Column	Type	Constraint	Description
history_id	UUID	PK, DEFAULT gen_random_uuid()	Unique identifier for the history record
batch_id	UUID	FK → credit_batch(batch_id), NOT NULL	Credit batch whose ownership changed
from_company_id	UUID	FK → company(company_id)	Previous owner (NULL on initial issuance)
to_company_id	UUID	FK → company(company_id)	New owner (NULL on retirement)
transaction_id	UUID	FK → transaction(transaction_id), NOT NULL	The transaction that triggered this change
quantity	INTEGER	NOT NULL, CHECK > 0	Number of credits transferred
transferred_at	TIMESTAMP	DEFAULT NOW()	Timestamp of ownership transfer

Note: ownership_history is an immutable insert-only table. No UPDATE or DELETE operations are permitted, preserving a tamper-proof audit trail.

Table 9: retirement_record

Column	Type	Constraint	Description
retirement_id	UUID	PK, DEFAULT gen_random_uuid()	Unique identifier for retirement record
batch_id	UUID	FK → credit_batch(batch_id), NOT NULL	Credit batch from which credits were retired
company_id	UUID	FK → company(company_id), NOT NULL	The company that retired the credits

Column	Type	Constraint	Description
transaction_id	UUID	FK → transaction(transaction_id), NOT NULL	Associated retirement transaction
quantity_retired	INTEGER	NOT NULL, CHECK > 0	Number of credits permanently retired
retirement_reason	VARCHAR(200)	—	Reason: COMPLIANCE, VOLUNTARY, or OFFSET
retired_at	TIMESTAMP	DEFAULT NOW()	Timestamp of retirement

Note: Once credits are retired, they cannot re-enter the market. The credit_batch status is updated to RETIRED when quantity_available reaches zero.

8. BUSINESS LOGIC IMPLEMENTED

8.1 Atomic Buy Transaction

The buy transaction is the most critical operation in the system and must be executed completely or not at all to maintain data consistency.

The process is carried out within a single database transaction block and includes the following steps:

1. Lock the selected credit batch using **SELECT FOR UPDATE** to prevent concurrent access.
2. Validate the availability of credits and ensure the buyer and seller are different.
3. Check if the buyer has sufficient wallet balance.
4. Insert a transaction record with status *COMPLETED*.
5. Debit the buyer's wallet and credit the seller's wallet.
6. Update the credit batch by reducing available quantity and transferring ownership.
7. Record the transfer in the ownership history table.
8. Commit the transaction to apply all changes atomically.

If any step fails, the transaction is rolled back, ensuring no partial updates occur.

8.2 Sell Listing

Selling operates on a listing mechanism where a company sets a price for its owned credit batch. This updates the unit price and marks the batch as available for purchase. No transaction is recorded at this stage; a transaction is created only when another company buys the credits.

8.3 Credit Retirement

Credit retirement permanently removes credits from circulation. The system records the retirement in the *retirement_record*, logs a corresponding transaction, updates ownership history, and reduces the available quantity. Once retired, credits cannot be traded again.

8.4 Double-Spend Prevention

To prevent double-spending, the system uses row-level locking (**SELECT FOR UPDATE**) on credit batches. This ensures that only one transaction can modify a batch at a time, maintaining data integrity during concurrent operations.

9. TESTING DOCUMENTS

The following are some of the validations done at the front end:

Table 9.1: Testing Documents

Field Name	Validations
Email (Login)	Accepts only a valid email format enforced by HTML5 input type=email. An empty submission is blocked by the required attribute.
Password (Login)	Required field — empty submission is blocked. Incorrect credentials show the "Invalid email or password" error message on screen.
Quantity (Buy Credits)	Accepts only positive integers. Minimum value is 1, enforced by min=1 on the number input. Cannot exceed available batch quantity enforced by the max attribute set dynamically via JavaScript.
Unit Price (Sell Credits)	Accepts only positive decimal values. Minimum value is 0.01 enforced by min=0.01 and step=0.01 on the number input. Zero or negative price is rejected.
Batch ID (Buy / Sell form)	Auto-filled by clicking the Buy or List for Sale button. Field is read-only to prevent manual typos or injection of invalid batch IDs.
Total Amount (Buy Credits)	Read-only field. Auto-calculated via JavaScript as quantity $\times$ unit price. Cannot be manually edited by the user.
Confirm Purchase button	Clicking without selecting a batch first leaves the Batch ID empty, and HTML5 required attribute blocks submission with an error.
Self-trade prevention	If a company attempts to buy its own credit batch, the system returns "You cannot buy your own credits" before any database operation is attempted.
Insufficient balance	If the total purchase amount exceeds the buyer's wallet balance, the system returns "Insufficient wallet balance," and no transaction is recorded.
Logout button	Clicking logout clears the session completely and redirects to the login page. Navigating back via browser after logout redirects to login due to a session check on all protected routes.

10. SCREEN SHOTS

Figure 10.1: Dashboard

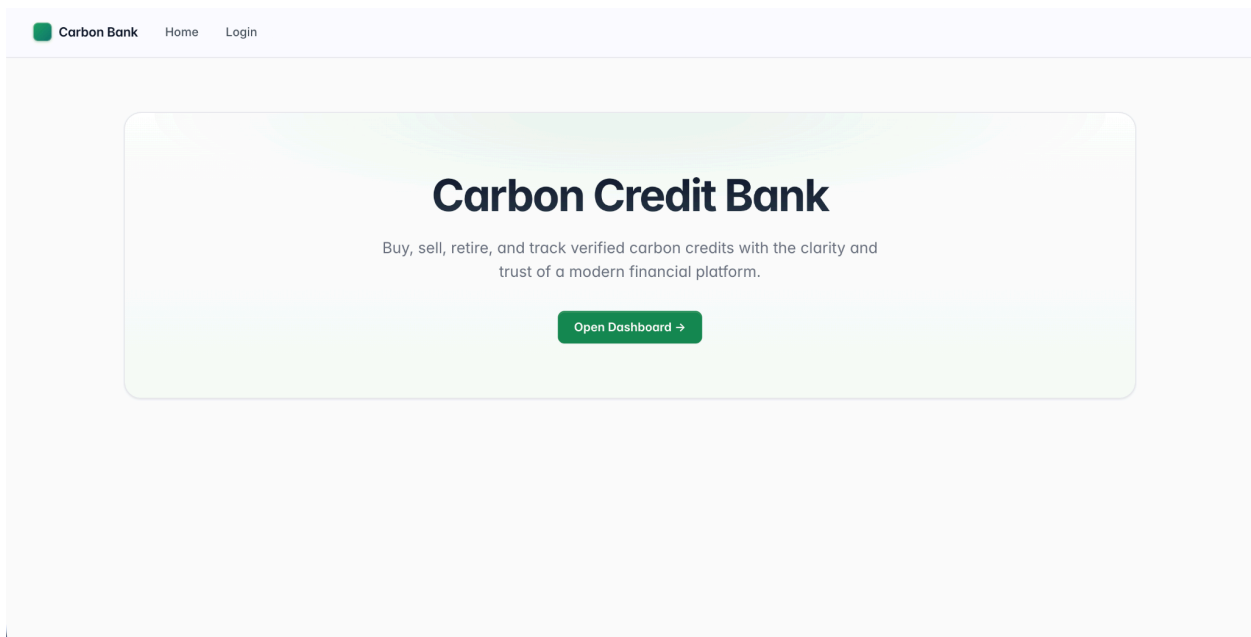


Figure 10.2: Login

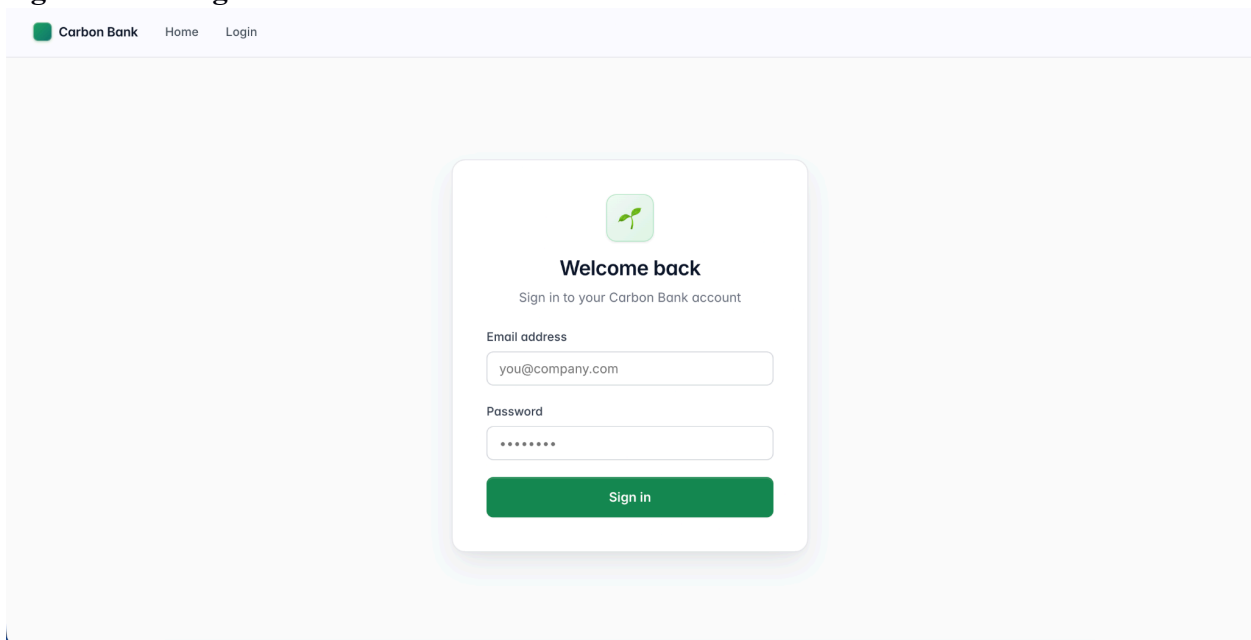


Figure 10.3: User dashboard

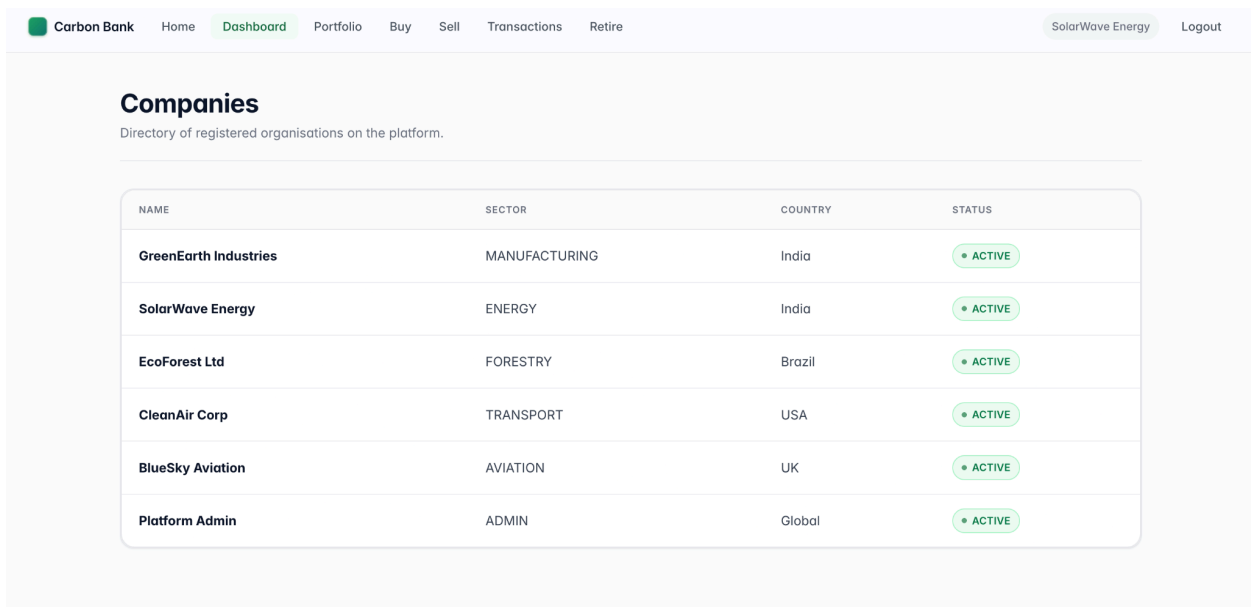


Figure 10.4: User Portfolio

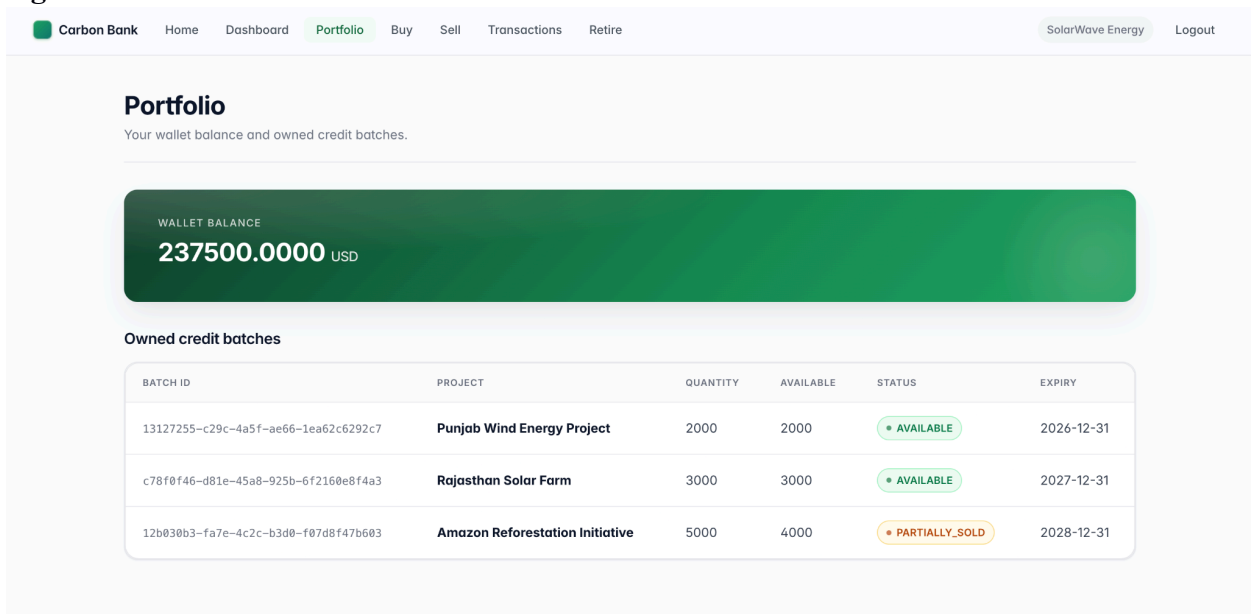


Figure 10.5: User buys credits

The screenshot shows the 'Buy credits' page of the Carbon Bank. The navigation bar at the top includes 'Carbon Bank', 'Home', 'Dashboard', 'Portfolio', 'Buy' (highlighted), 'Sell', 'Transactions', and 'Retire'. On the right, there are links for 'SolarWave Energy' and 'Logout'. The main heading is 'Buy credits' with a subtext: 'Select a batch and the quantity you want to purchase.' Below this, there is a form with two input fields: 'Select batch' (containing 'Punjab Wind Energy Project — 2500 available @ 11.0000 USD') and 'Quantity' (empty). A green button labeled 'Buy credits' is at the bottom of the form.

Carbon Bank Home Dashboard Portfolio **Buy** Sell Transactions Retire SolarWave Energy Logout

Buy credits

Select a batch and the quantity you want to purchase.

Select batch

Punjab Wind Energy Project — 2500 available @ 11.0000 USD

Quantity

Buy credits

Figure 10.6: User sells credits

The screenshot shows the 'Sell credits' page of the Carbon Bank. The navigation bar at the top includes 'Carbon Bank', 'Home', 'Dashboard', 'Portfolio', 'Buy', 'Sell' (highlighted), 'Transactions', and 'Retire'. On the right, there are links for 'SolarWave Energy' and 'Logout'. The main heading is 'Sell credits' with a subtext: 'List an owned batch on the marketplace at your chosen price.' Below this, there is a form with two input fields: 'Select owned batch' (containing 'Rajasthan Solar Farm — 3000 available') and 'Price per credit' (empty). A green button labeled 'List for sale' is at the bottom of the form.

Carbon Bank Home Dashboard Portfolio Buy **Sell** Transactions Retire SolarWave Energy Logout

Sell credits

List an owned batch on the marketplace at your chosen price.

Select owned batch

Rajasthan Solar Farm — 3000 available

Price per credit

List for sale

Figure 10.7: User transaction

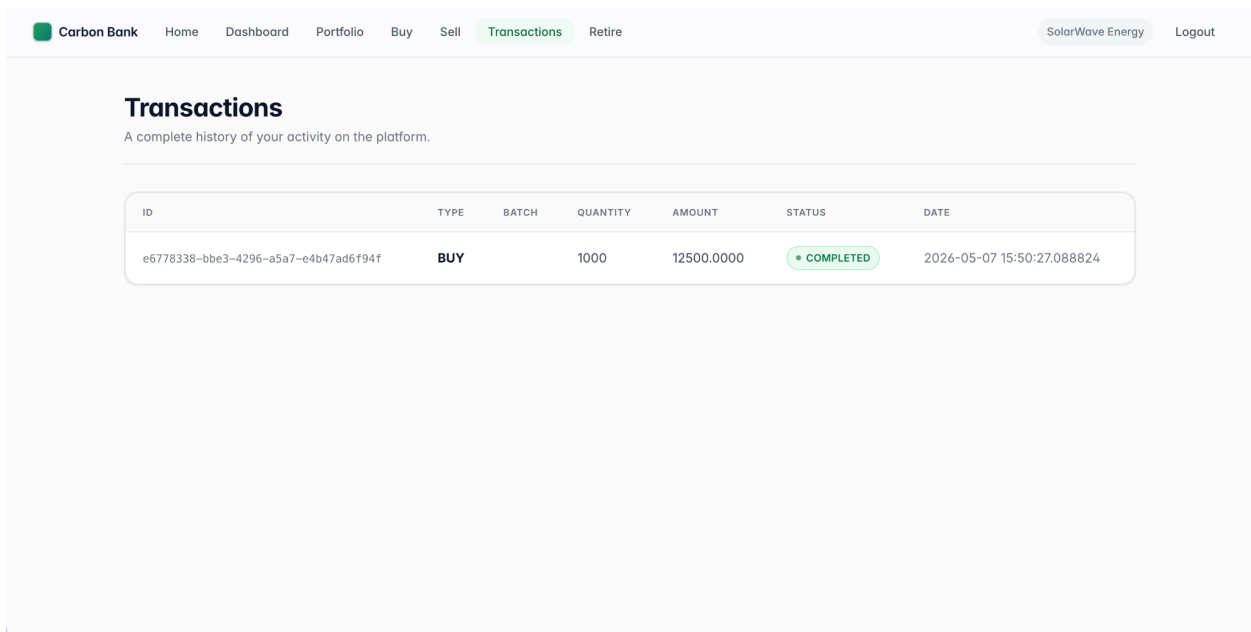


Figure 10.8: Admin dashboard

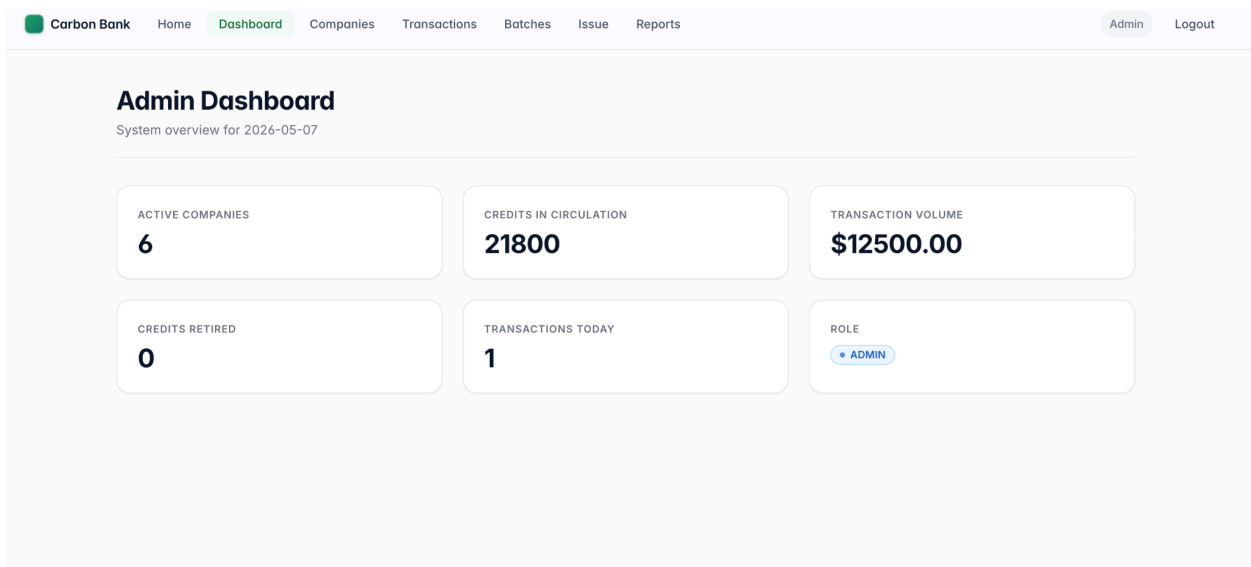


Figure 10.9: Admin company list

Carbon Bank

HomeDashboardCompaniesTransactionsBatchesIssueReports

AdminLogout

Company Management

Suspend, reactivate, or dissolve companies (dissolve requires zero balance and credits).

NAME	SECTOR	COUNTRY	STATUS	WALLET	ACTIONS
GreenEarth Industries	MANUFACTURING	India	ACTIVE	100000.00 USD	SuspendReactivateDissolve
CleanAir Corp	TRANSPORT	USA	ACTIVE	320000.00 USD	SuspendReactivateDissolve
BlueSky Aviation	AVIATION	UK	ACTIVE	95000.00 USD	SuspendReactivateDissolve
SolarWave Energy	ENERGY	India	ACTIVE	237500.00 USD	SuspendReactivateDissolve
EcoForest Ltd	FORESTRY	Brazil	ACTIVE	192500.00 USD	SuspendReactivateDissolve
Platform Admin	ADMIN	Global	ACTIVE	0.00 USD	SuspendReactivateDissolve

Figure 10.10: Admin transaction list

Carbon Bank

HomeDashboardCompaniesTransactionsBatchesIssueReports

AdminLogout

All Transactions

System-wide transaction ledger with simple filters.

Type

BUY / SELL / ISSUANCE / RETIREMENT

Status

COMPLETED / PENDING / FAILED

Date Range

dd/mm/yyyy

dd/mm/yyyy

Apply Filters

Clear

ID	BUYER	SELLER	PROJECT	QTY	TOTAL	TYPE	STATUS	TIMESTAMP
e6778338-bbe3-4296-a5a7-e4b47ad6f94f	SolarWave Energy	EcoForest Ltd	Amazon Reforestation Initiative	1000	\$12500.00	BUY	COMPLETED	2026-05-07 15:50:27.088824

Figure 10.11: Admin registry list

Carbon Bank

HomeDashboardCompaniesTransactionsBatchesIssueReports

AdminLogout

Credit Batch Registry

View all batches and mark expired batches if needed.

BATCH	OWNER	PROJECT	REGISTRY	QTY	PRICE	EXPIRY	STATUS	ACTION
c78f0f46-d81e-45a8-925b-6f2160e8f4a3	SolarWave Energy	Rajasthan Solar Farm (SOLAR)	Gold Standard	3000	\$15.00	2027-12-31	AVAILABLE	Mark Expired
3acbe044-e3e3-4a1e-a1c8-70a8c5b11816	GreenEarth Industries	Maharashtra Methane Capture (METHANE_CAPTURE)	Verra Registry	4000	\$18.00	2029-12-31	AVAILABLE	Mark Expired
73661f3d-3ce1-4f41-af36-f964bfc8339	BlueSky Aviation	Amazon Reforestation Initiative (REFORESTATION)	Verra Registry	3500	\$14.50	2028-05-31	AVAILABLE	Mark Expired
166694bf-823e-414f-b146-85cd371367b5	CleanAir Corp	Rajasthan Solar Farm (SOLAR)	Gold Standard	2800	\$16.00	2027-02-28	AVAILABLE	Mark Expired

Figure 10.12: Admin credit issue

Carbon Bank

HomeDashboardCompaniesTransactionsBatchesIssueReports

AdminLogout

Issue New Credit Batch

Create new credits (ISSUANCE) and assign them to a company.

Project

Amazon Reforestation Initiative

Registry

American Carbon Registry

Assign to Company

BlueSky Aviation

Certification Standard

VCS / Gold Standard / ACR

Quantity

Unit Price

Vintage Year

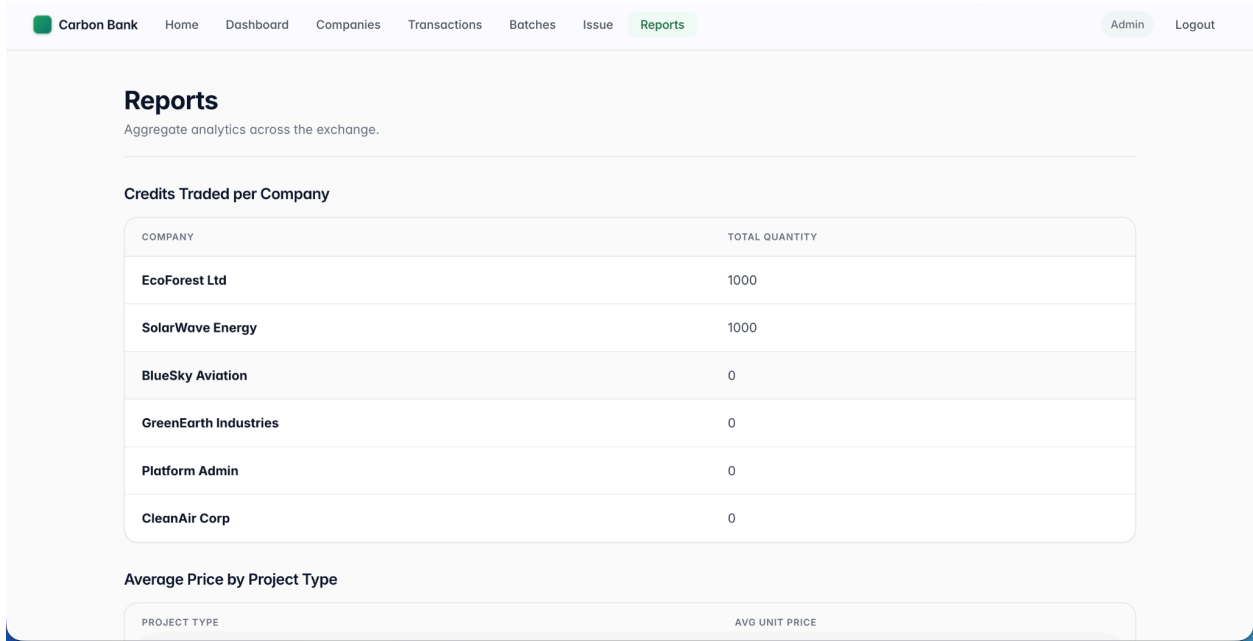
dd/mm/yyyy

Expiry Date (optional)

dd/mm/yyyy

Issue Credits

Figure 10.13: Admin report



CONCLUSION

The Carbon Credit Bank project successfully demonstrates the design and implementation of a complete, normalized, transactional DBMS application. From entity-relationship modeling and normalization to a fully functional web application, every stage of the database development lifecycle was applied.

The project shows how a well-designed relational schema with proper constraints, indexes, and transactional logic can enforce complex business rules — double-spend prevention, audit immutability, and atomic wallet operations — entirely at the database layer. The choice of PostgreSQL over simpler alternatives was justified by the need for enforced CHECK constraints and row-level locking, which are essential for a financially sensitive application.

The application is fully functional with login, portfolio management, buying and selling of credits, transaction history, and credit retirement, all working end-to-end. The system is accessible to all team members via a GitHub repository that includes complete setup instructions for both macOS and Windows.

Future enhancements could include a role-based admin analytics dashboard, a REST API layer for external integrations, password hashing for production security, and automated testing with CI/CD integration.

REFERENCES

1. Flask Documentation — <https://flask.palletsprojects.com/>
2. PostgreSQL 16 Documentation — <https://www.postgresql.org/docs/16/>
3. psycopg2 Documentation — <https://www.psycopg.org/docs/>
4. Jinja2 Template Designer Documentation — <https://jinja.palletsprojects.com/>
5. Verra Registry — Carbon Credit Standards — <https://verra.org/>
6. Gold Standard Foundation — <https://www.goldstandard.org/>
7. Elmasri R., Navathe S.B. — Fundamentals of Database Systems, 7th Edition, Pearson
8. Ramakrishnan R., Gehrke J. — Database Management Systems, 3rd Edition, McGraw-Hill